Documento de Engenharia de Software

Feature Request & Product Roadmap Management System

EDS-13 — Diagrama de Classes e Decisões de Design

Ano Letivo 2026/27

Índice

1 Objetivo **2**

2 Visão geral do modelo **2**

3 Decisões de design **3**

3.1 Modelo de domínio, com as operações essenciais 3
3.2 Separar Pedido de User Story 3
3.3 Perfil como atributo, não como subclasses 3
3.4 Estados como enumerações 3
3.5 Voto como classe própria 4
3.6 Composição para Anexo e Critério de Aceitação 4
3.7 Um só Comentario para pedidos e User Stories 4
3.8 Dependências entre User Stories 4
3.9 Datas e capacidade para as métricas 4
3.10 Product Backlog sem classe própria 5
3.11 Roadmap ao nível do Pedido 5
3.12 Categoria opcional 5
3.13 Justificação da rejeição e equipas 5
3.14 Etiquetas (tags) e fusão de duplicados 6
3.15 O que fica de fora nesta versão 6

4 Ligação aos outros artefactos **6**

1 Objetivo

Este documento apresenta o diagrama de classes do domínio do sistema e explica as decisões de modelação tomadas. O diagrama descreve as entidades que o sistema guarda, os seus atributos, as operações principais e as relações entre elas. Serve de base aos modelos Django (`models.py`) e ao modelo entidade-relação da base de dados (EDS-14).

O diagrama foi construído a partir das User Stories do documento de Personas. Cada classe, atributo ou operação existe porque pelo menos uma User Story precisa dele; essa ligação está indicada em cada decisão.

2 Visão geral do modelo

O diagrama está a seguir às decisões, em formato vetorial e numa página maior do que A4, e pode ser ampliado sem perder qualidade. O código PlantUML que o gera está nas últimas páginas e pode ser colado em `planttext.com` para o editar.

O modelo tem 14 classes e 3 enumerações, organizadas nas áreas dos módulos do back-end (EDS-10):

- **Utilizadores e equipas** (`contas`): Utilizador e Equipa.
- **Pedidos e priorização** (`pedidos`, `priorizacao`): Pedido, Categoria, Tag, Anexo e Voto.
- **Colaboração** (`colaboracao`): Comentario e Notificacao.
- **Backlog e requisitos** (`backlog`, `requisitos`): UserStory, CriterioAceitacao e Sprint.
- **Roadmap** (`roadmap`): Release e Milestone.
- **Enumerações:** Perfil, EstadoPedido e EstadoUserStory.

O percurso principal lê-se assim: um Utilizador submete um Pedido; os clientes votam e comentam; quando aprovado, o Pedido origina uma ou mais User Stories (que também podem ser criadas manualmente, sem pedido), cada uma com Critérios de Aceitação; as User Stories entram num Sprint e o Pedido é incluído numa Release.

3 Decisões de design

3.1 Modelo de domínio, com as operações essenciais

Decisão. O diagrama mostra os atributos com tipo e visibilidade e apenas as operações que exprimem regras de negócio (por exemplo `aprovar()`, `estaPronta()`, `cycleTime()`). Getters, setters e construtores não são representados.

Porquê. Nesta fase o objetivo é fixar o vocabulário e a estrutura do domínio para toda a equipa. As operações escolhidas são as que aparecem nos diagramas de sequência e de estados, o que mantém os diagramas coerentes entre si.

Alternativa considerada. Um diagrama de implementação completo, com todos os métodos. Foi adiado para quando o código existir, porque mudaria a cada sprint.

3.2 Separar Pedido de User Story

Decisão. O Pedido (o que o cliente pede, na linguagem dele) e a UserStory (o trabalho de engenharia) são classes diferentes. Um Pedido origina zero ou mais User Stories; cada User Story nasce de zero ou um Pedido (multiplicidade `0..1`).

Porquê. São conceitos com ciclos de vida e utilizadores diferentes: a Sofia vê o Pedido, o Rui trabalha na User Story. Um pedido grande pode ser dividido em várias stories. A ligação entre os dois garante a rastreabilidade pedida pela US 3.1 ("Origem do Pedido") e a geração automática da US 1.3. A US 3.1 prevê também stories criadas manualmente, sem pedido associado ("Sem pedido de origem"), por isso a ligação ao Pedido é opcional.

Alternativa considerada. Uma única classe com todos os campos, que misturaria dados do cliente com dados técnicos e impediria dividir um pedido em várias stories; ou ligação obrigatória (`1`), que impediria as stories criadas manualmente.

3.3 Perfil como atributo, não como subclasses

Decisão. O tipo de utilizador (Cliente, Gestor de Produto, Programador, Gestor de Projeto) é o atributo `perfil` do Utilizador, do tipo enumerado `Perfil`.

Porquê. Os quatro perfis têm os mesmos dados e só diferem nas permissões, que são tratadas na camada de autenticação. Um atributo é mais simples de guardar e permite mudar o perfil de uma pessoa sem recriar o objeto.

Alternativa considerada. Herança (Cliente, Programador, etc. como subclasses de Utilizador). Só se justificaria se cada perfil tivesse atributos próprios, o que não acontece.

3.4 Estados como enumerações

Decisão. O estado do Pedido e o estado da UserStory são enumerações (`EstadoPedido` e `EstadoUserStory`) com os mesmos valores do Glossário de Estados das Personas.

Porquê. Os estados são um conjunto fechado e conhecido. A enumeração impede valores inválidos e faz a ponte direta com os diagramas de estados, onde cada valor é um estado e cada operação (`aprovar()`, `iniciar()`) é uma transição.

Alternativa considerada. Uma classe Estado separada, ou texto livre. A primeira acrescenta complexidade sem benefício; o segundo permite erros de escrita.

3.5 Voto como classe própria

Decisão. Cada voto é um objeto Voto que liga um Utilizador a um Pedido e guarda a data. Um utilizador só pode votar uma vez em cada pedido (nota no diagrama).

Porquê. Guardar quem votou permite três coisas exigidas pela US 2.2 e pela US 2.3: impedir votos repetidos, retirar o voto e notificar quem votou quando o pedido muda de estado. O total de votos é calculado por `totalVotos()`.

Alternativa considerada. Um simples contador inteiro no Pedido. Não permitiria saber quem votou, retirar votos nem enviar notificações.

3.6 Composição para Anexo e Critério de Aceitação

Decisão. Anexo pertence ao Pedido e CriterioAceitacao pertence à UserStory através de composição (losango preto). Uma User Story tem zero ou mais critérios (multiplicidade `0..*`).

Porquê. Estes objetos não têm significado fora do dono: apagar o pedido apaga os anexos, apagar a story apaga os critérios. Uma story pode existir no Product Backlog sem critérios (por exemplo, quando a IA falha ao gerá-los); a Definition of Ready da US 3.2, que exige pelo menos um critério, só é aplicada quando a story entra num Sprint, através de `estaPronta()`. Pôr `1..*` no diagrama impediria criar essas stories.

Alternativa considerada. Associação simples, que não comunicaria a dependência de existência; ou multiplicidade `1..*`, que contradiria o fluxo de geração de User Stories.

3.7 Um só Comentario para pedidos e User Stories

Decisão. A classe Comentario está ligada a Pedido e a UserStory, ambas com multiplicidade `0..1`, com a restrição de pertencer exatamente a um dos dois.

Porquê. Os comentários têm a mesma estrutura nos dois contextos. A Sofia comenta pedidos e o Rui comenta User Stories (cenário da US 3.3); uma só classe evita duplicar código e ecrãs.

Alternativa considerada. Duas classes (ComentarioPedido e ComentarioStory) com os mesmos atributos, o que duplicaria o modelo.

3.8 Dependências entre User Stories

Decisão. A UserStory tem uma auto-associação "depende de" de muitos para muitos, o atributo `dataPrevista` e a operação `estaBloqueada()`.

Porquê. A US 3.3 exige assinalar uma dependência em risco (amarelo) quando faltam 2 dias ou menos para a data prevista da story de que se depende, e bloqueada (vermelho) quando essa data passa sem estar concluída. A auto-associação guarda as ligações, `dataPrevista` guarda a data de referência e `estaBloqueada()` aplica a regra.

Alternativa considerada. Um campo de texto com as dependências. Não permitiria calcular bloqueios automaticamente.

3.9 Datas e capacidade para as métricas

Decisão. O Pedido guarda `dataSubmissao` e `dataLancamento`; a UserStory guarda `dataInicio` e `dataConclusao`; o Utilizador guarda `capacidadeSP`.

Porquê. São os dados mínimos para as métricas da Inês: o Lead Time vem das datas do Pedido, o Cycle Time das datas da User Story (US 4.1), e a capacidade permite o alerta de sobrecarga

acima de 80% (US 4.2) através de `cargaNoSprint()`.

Alternativa considerada. Calcular as métricas a partir de um histórico de eventos. É mais flexível, mas mais pesado do que o necessário para este projeto.

3.10 Product Backlog sem classe própria

Decisão. O Product Backlog é o conjunto das User Stories que ainda não estão num Sprint (associação `0..1` com Sprint), ordenadas pelo atributo `ordemBacklog`. O Sprint Backlog são as User Stories ligadas a um Sprint.

Porquê. Uma classe ProductBacklog só teria uma lista de stories, sem dados próprios. Com o atributo de ordem, o Product Owner pode reordenar o backlog (priorização manual) sem criar entidades extra.

Alternativa considerada. Uma classe ProductBacklog ou BacklogItem. Acrescentaria uma camada sem informação nova.

3.11 Roadmap ao nível do Pedido

Decisão. A Release agrega Pedidos (cada Pedido está em zero ou uma Release) e é composta por Milestones.

Porquê. O roadmap é apresentado à direção e aos clientes em termos de funcionalidades (pedidos), não de tarefas técnicas, como pede a "Visão Executiva" da US 1.1. A ligação do Pedido à Release também dispara a transição para o estado Planeado.

Alternativa considerada. Ligar User Stories diretamente a Releases. Exporia detalhe técnico no roadmap e complicaria a Visão Executiva.

3.12 Categoria opcional

Decisão. Cada Pedido pertence a zero ou uma Categoria.

Porquê. O cliente não escolhe a categoria ao submeter (o formulário da US 2.1 só tem dois campos); a categoria é sugerida pela IA e confirmada pelo Gestor de Produto na triagem (US 1.2).

Alternativa considerada. Categoria obrigatória. Obrigaria o cliente a escolher, contrariando a US 2.1.

3.13 Justificação da rejeição e equipas

Decisão. O Pedido guarda a `justificacaoRejeicao` e `rejeitar()` recebe-a como parâmetro. A ligação entre Equipa e Utilizador é de muitos para muitos.

Porquê. O Gestor de Produto rejeita sempre com justificação (Diagrama de Sequência 1), que fica registada para consulta. Uma pessoa pode pertencer a várias equipas, por exemplo um programador partilhado entre duas equipas; no modelo ER esta ligação é a tabela `equipa_membro`.

Alternativa considerada. Rejeitar sem justificação, ou cada utilizador numa única equipa. A primeira perdia a razão da decisão; a segunda não corresponde à forma como as equipas se organizam.

3.14 Etiquetas (tags) e fusão de duplicados

Decisão. A classe Tag liga-se ao Pedido numa associação de muitos para muitos. O Pedido tem a operação `fundirCom(original)`.

Porquê. O Product Backlog prevê para o Sprint 3 a categorização manual com etiquetas e a pesquisa por tag; ao contrário da categoria (uma por pedido), um pedido pode ter várias etiquetas e cada etiqueta aparece em vários pedidos. A fusão de duplicados (US 1.5, Diagrama de Sequência 1) transfere os votos e anexos para o pedido original e termina o ciclo de vida do duplicado, como mostra o diagrama de estados.

Alternativa considerada. Guardar as etiquetas num campo de texto separado por vírgulas, que impediria filtrar com eficiência; ou um estado `DUPLICADO`, que manteria o pedido repetido no sistema e contaria os votos em duplicado.

3.15 O que fica de fora nesta versão

Decisão. O Serviço de IA, o histórico de versões dos pedidos e os links de partilha do roadmap não aparecem como classes. As sugestões da IA são guardadas no atributo `sugestoesIA` do Pedido, em formato JSON, sem classe própria.

Porquê. O Serviço de IA é um sistema externo (aparece no diagrama C4, EDS-11), não uma entidade do domínio. As sugestões são dados temporários de apoio à triagem, sem regras próprias, por isso um campo JSON chega. O histórico de versões e os links de partilha estão previstos para o Sprint 5 no Product Backlog; serão acrescentadas ao diagrama quando forem desenhadas em detalhe, para manter o modelo legível agora.

Alternativa considerada. Incluir já todas as classes. Foi experimentado numa versão anterior com mais de 20 classes, que se tornou difícil de ler e de explicar.

4 Ligação aos outros artefactos

- **Personas e User Stories:** cada decisão acima indica a User Story que a justifica.
- **Diagramas de sequência e de estados:** usam as mesmas classes, estados e operações (`aprovar()`, `gerarUserStory()`, `iniciar()`).
- **Modelo entidade-relação (EDS-14):** cada classe dá origem a uma tabela; as associações de muitos para muitos (membros de equipa, etiquetas e dependências) dão origem a tabelas de associação.
- **Implementação:** cada classe corresponde a um modelo Django, e as enumerações a campos com `choices`.

Diagrama de Classes

Utilizador
-nome: String
-email: String
-perfil: Perfil
-capacidadeSP: int
+cargaNoSprint(s: Sprint): float

Equipa
-nome: String

Notificacao
-mensagem: String
-lida: boolean
-data: DateTime

Pedido
-titulo: String
-problema: String
-estado: EstadoPedido
-esforco: int
-impacto: int
-dataSubmissao: DateTime
-dataLancamento: DateTime
-justificacaoRejeicao: String
-sugestoesIA: JSON
+aprovar(): void
+rejeitar(justificacao: String): void
+gerarUserStory(): UserStory
+totalVotos(): int
+fundirCom(original: Pedido): void

Categoria
-nome: String

Voto
-data: DateTime

Um utilizador só pode votar uma vez em cada pedido

Sprint
-numero: int
-objetivo: String
-dataInicio: Date
-dataFim: Date
+velocidade(): int

UserStory
-descricao: String
-storyPoints: int
-ordemBacklog: int
-dataPrevista: Date
-estado: EstadoUserStory
-dataInicio: DateTime
-dataConclusao: DateTime
+iniciar(): void
+estaPronta(): boolean
+estaBloqueada(): boolean
+cycleTime(): int

Anexo
-nomeFicheiro: String
-url: String

Release
-nome: String
-dataPrevista: Date
+progresso(): float

Milestone
-nome: String
-data: Date

Tag
-nome: String

CriterioAceitacao
-dado: String
-quando: String
-entao: String

Definition of Ready: estaPronta() exige pelo menos um critério

Perfil
CLIENTE
GESTOR_PRODUTO
PROGRAMADOR
GESTOR_PROJETO

EstadoPedido
SUBMETIDO
EM_TRIAGEM
APROVADO
REJEITADO
PLANEADO
EM_DESENVOLVIMENTO
LANCADO

EstadoUserStory
A_FAZER
EM_PROGRESSO
EM_REVISAO
CONCLUIDO

Comentario
-texto: String
-data: DateTime

Pertence a um Pedido ou a uma UserStory

tem membros
recebe
planeia
submete
dá
responsável por
pertence a
recebe
contém
depende de
origina
tem
inclui
tem etiquetas
tem
escreve
tem
tem

Código PlantUML

```
@startuml classes_mao
skinparam shadowing false
skinparam handwritten true
skinparam backgroundColor #FFFFFF
skinparam defaultFontName Kalam
skinparam defaultFontSize 16
skinparam dpi 150
skinparam classAttributeIconSize 0
skinparam nodesep 60
skinparam ranksep 80
skinparam note {
  BackgroundColor #FFFFFF
  BorderColor #1F2A44
}
skinparam class {
  BackgroundColor White
  BorderColor #1F2A44
  FontColor #1F2A44
  AttributeFontColor #1F2A44
  ArrowColor #1F2A44
}
hide circle
hide empty members

title Diagrama de Classes

class Utilizador {
  - nome: String
  - email: String
  - perfil: Perfil
  - capacidadeSP: int
  + cargaNoSprint(s: Sprint): float
}
class Equipa {
  - nome: String
}
class Notificacao {
  - mensagem: String
  - lida: boolean
  - data: DateTime
}

class Pedido {
  - titulo: String
  - problema: String
  - estado: EstadoPedido
  - esforco: int
  - impacto: int
  - dataSubmissao: DateTime
  - dataLancamento: DateTime
  - justificacaoRejeicao: String
  - sugestoesIA: JSON
  + aprovar(): void
  + rejeitar(justificacao: String): void
  + gerarUserStory(): UserStory
  + totalVotos(): int
  + fundirCom(original: Pedido): void
}
class Anexo {
  - nomeFicheiro: String
  - url: String
```

```
}
class Categoria {
  - nome: String
}
class Tag {
  - nome: String
}
class Voto {
  - data: DateTime
}
class Comentario {
  - texto: String
  - data: DateTime
}

class UserStory {
  - descricao: String
  - storyPoints: int
  - ordemBacklog: int
  - dataPrevista: Date
  - estado: EstadoUserStory
  - dataInicio: DateTime
  - dataConclusao: DateTime
  + iniciar(): void
  + estaPronta(): boolean
  + estaBloqueada(): boolean
  + cycleTime(): int
}
class CriterioAceitacao {
  - dado: String
  - quando: String
  - entao: String
}
class Sprint {
  - numero: int
  - objetivo: String
  - dataInicio: Date
  - dataFim: Date
  + velocidade(): int
}
class Release {
  - nome: String
  - dataPrevista: Date
  + progresso(): float
}
class Milestone {
  - nome: String
  - data: Date
}

enum Perfil {
  CLIENTE
  GESTOR_PRODUTO
  PROGRAMADOR
  GESTOR_PROJETO
}
enum EstadoPedido {
  SUBMETIDO
  EM_TRIAGEM
  APROVADO
  REJEITADO
  PLANEADO
  EM_DESENVOLVIMENTO
```

```
  LANCADO
}
enum EstadoUserStory {
  A_FAZER
  EM_PROGRESSO
  EM_REVISAO
  CONCLUIDO
}

' pessoas
Equipa "*" -right- "*" Utilizador : tem membros >
Utilizador "1" -right- "*" Notificacao : recebe >

' pedidos
Utilizador "1" -down- "*" Pedido : submete >
Utilizador "1" -down- "*" Voto : dá >
Pedido "1" -right- "*" Voto : recebe >
Pedido "*" -left- "0..1" Categoria : pertence a >
Pedido "*" -- "*" Tag : tem etiquetas >
Pedido "1" *-- "*" Anexo : tem >
Utilizador "1" -- "*" Comentario : escreve >
Pedido "0..1" -- "*" Comentario : tem >
UserStory "0..1" -- "*" Comentario : tem >

' desenvolvimento
Pedido "0..1" -down- "*" UserStory : origina >
UserStory "1" *-down- "0..*" CriterioAceitacao : tem >
UserStory "*" -- "*" UserStory : depende de >
Sprint "0..1" -right- "*" UserStory : contém >
Equipa "1" -down- "*" Sprint : planeia >
Utilizador "0..1" -- "*" UserStory : responsável por >

' roadmap
Release "0..1" -up- "*" Pedido : inclui >
Release "1" *-right- "*" Milestone : tem >

note right of Voto
  Um utilizador só pode votar
  uma vez em cada pedido
end note
note right of CriterioAceitacao
  Definition of Ready: estaPronta()
  exige pelo menos um critério
end note
note right of Comentario
  Pertence a um Pedido
  ou a uma UserStory
end note

' enumerações ao fundo
Anexo -[hidden]down- Perfil
Perfil -[hidden]right- EstadoPedido
EstadoPedido -[hidden]right- EstadoUserStory
@enduml
```